Linux File Permissions

A Complete Guide to chmod, chown, umask & Access Control

Covers the permission model, ls -l output, chmod (symbolic & octal), chown/chgrp, umask, special permissions (SUID, SGID, Sticky Bit), and Access Control Lists (ACLs) with practical examples.

1. Understanding the Linux Permission Model

Every file and directory in Linux has an owner, a group, and a set of permissions that control who can read, write, or execute it. Permissions are checked in a strict order: **owner** permissions apply if you are the file's owner, otherwise **group** permissions apply if you belong to the file's group, otherwise **others** permissions apply to everyone else.

1.1 Viewing Permissions with ls -l

```
$ ls -l notes.txt
-rwxr-xr-- 1 pratik devs 4096 Aug 10 09:15 notes.txt
```

This output breaks down into distinct fields:

Field	Example	Meaning
File type	-	- = regular file, d = directory, l = symlink
Owner perms	rwx	Read, write, execute for the owner
Group perms	r-x	Read, execute (no write) for the group
Other perms	r--	Read only for everyone else
Link count	1	Number of hard links
Owner	pratik	User who owns the file
Group	devs	Group that owns the file
Size	4096	File size in bytes

1.2 The Three Permission Types

Symbol	Permission	Effect on a File	Effect on a Directory
r	Read	View file contents	List directory contents (ls)
w	Write	Modify/delete contents	Create/delete/rename files inside
x	Execute	Run as a program/script	Enter the directory (cd) / access files

Note: Execute on a directory is what actually lets you 'cd' into it or access files inside by full path — without it, even knowing a filename won't let you open the file, regardless of read permission on that directory.

2. chmod — Changing Permissions (Symbolic Mode)

chmod (change mode) modifies permissions. Symbolic mode uses letters, making intent easy to read.

Who	Symbol
User/owner	u
Group	g

Others	o
All (u+g+o)	a

Operator	Meaning
+	Add a permission
-	Remove a permission
=	Set exactly (overwrite existing)

2.1 Common Examples

```
$ chmod u+x script.sh # add execute for owner
$ chmod g-w report.docx # remove write for group
$ chmod o=r file.txt # set 'others' to read-only exactly
$ chmod a+r public.txt # add read for everyone
$ chmod u+x,g+x,o-w deploy.sh # multiple changes, comma separated
$ chmod -R g+w project/ # apply recursively to a directory tree
```

Note: -R applies chmod recursively to every file and subdirectory. Use carefully — it can open up permissions further than intended, especially with 'a' or 'o'.

3. chmod — Changing Permissions (Octal / Numeric Mode)

Octal mode represents rwx as a single digit per group (owner, group, other) by summing values: r = 4, w = 2, x = 1. Add the values you want to enable.

Digit	Permissions	Symbolic
7	read + write + execute	rwx
6	read + write	rw-
5	read + execute	r-x
4	read only	r--
3	write + execute	-wx
2	write only	-w-
1	execute only	--x
0	no permissions	---

3.1 Building a Full Permission Set

A full mode is three digits: owner, group, other. For example, 750 means:

- **7** (owner) → rwx — full control
- **5** (group) → r-x — read and execute, no write

- **0** (other) → --- — no access at all

```
$ chmod 750 script.sh # rwxr-x---
$ chmod 644 document.txt # rw-r--r-- (typical file default)
$ chmod 755 program # rwxr-xr-x (typical executable/script)
$ chmod 700 private_key # rwx------ (owner-only access)
$ chmod -R 755 website/ # recursively set a whole directory tree
```

3.2 Quick Reference: Common Combinations

Octal	Meaning	Typical Use
644	rw-r--r--	Regular files (documents, configs)
664	rw-rw-r--	Group-shared editable files
600	rw-------	Private files (SSH keys, secrets)
755	rwxr-xr-x	Executables, scripts, public directories
700	rwx------	Private directories
777	rwxrwxrwx	Full access to all — avoid on real systems

Note: Avoid 777. It grants write access to everyone, including other users on the system, which is a common security misconfiguration.

4. chown & chgrp — Changing Ownership

chown changes the file's owner (and optionally group); chgrp changes only the group. Both usually require root/sudo privileges when changing to another user.

```
$ chown pratik file.txt # change owner to 'pratik'
$ chown pratik:devs file.txt # change owner AND group
$ chown :devs file.txt # change group only (via chown)
$ chgrp devs file.txt # change group only (via chgrp)
$ chown -R pratik:devs project/ # recursively change ownership
$ sudo chown root:root /etc/passwd # typical system-file ownership
```

Note: Only root (or the current owner, in limited cases) can change a file's owner to someone else. Regular users generally can only change the group of files they own, and only to a group they belong to.

5. umask — Default Permission Masks

umask determines the default permissions new files and directories get when created. It works by 'subtracting' permission bits from the system defaults (666 for files, 777 for directories).

```
$ umask # view current mask, e.g. 0022
$ umask 022 # set the mask for this shell session
$ umask 027 # stricter mask (no access for 'others')
```

5.1 How the Math Works

Item	Base Default	umask 022	Resulting Perms

New file	666 (rw-rw-rw-)	subtract 022	644 (rw-r--r--)
New directory	777 (rwxrwxrwx)	subtract 022	755 (rwxr-xr-x)

Note: Files never get execute permission by default, even with umask 000 — that's a base OS rule, not a umask rule. You must chmod +x explicitly.

To make umask persistent, add the umask command to ~/.bashrc, ~/.profile, or /etc/profile for a system-wide default.

6. Special Permissions: SUID, SGID, Sticky Bit

6.1 SUID (Set User ID) — Files

When set on an executable, SUID makes the program run with the file owner's privileges, not the privileges of the user running it. Classic example: /usr/bin/passwd runs as root so any user can update the password file.

```
$ chmod u+s program # symbolic: add SUID
$ chmod 4755 program # octal: leading 4 = SUID
$ ls -l program
-rwsr-xr-x 1 root root ... program # 's' replaces owner's 'x'
```

6.2 SGID (Set Group ID) — Files and Directories

On an executable, SGID runs the program with the file's group privileges. On a directory, SGID makes new files/subdirectories created inside automatically inherit the directory's group — very useful for shared team folders.

```
$ chmod g+s /shared/team_folder # symbolic
$ chmod 2775 /shared/team_folder # octal: leading 2 = SGID
$ ls -ld /shared/team_folder
drwxrwsr-x 2 root devs ... team_folder # 's' replaces group's 'x'
```

6.3 Sticky Bit — Directories

On a directory, the sticky bit restricts deletion: users can only delete or rename files they personally own, even if they have write access to the directory. /tmp is the textbook example.

```
$ chmod +t /shared/dropbox # symbolic: add sticky bit
$ chmod 1777 /shared/dropbox # octal: leading 1 = sticky bit
$ ls -ld /tmp
drwxrwxrwt 10 root root ... /tmp # 't' replaces other's 'x'
```

6.4 Special Permission Octal Summary

Leading Digit	Bit	Symbolic Flag	Shown As
4	SUID	u+s	rws (or rwS if no owner-execute)
2	SGID	g+s	rws (or rwS if no group-execute)
1	Sticky Bit	+t	rwt (or rwT if no other-execute)

Note: A capital S or T (instead of lowercase s/t) means the special bit is set but the underlying execute bit is NOT — usually a sign of a misconfiguration.

7. Access Control Lists (ACLs) — Beyond owner/group/other

Standard permissions only support one owner and one group. ACLs let you grant fine-grained permissions to additional specific users or groups on the same file.

```
$ setfacl -m u:alice:rw file.txt # give user 'alice' read+write
$ setfacl -m g:interns:r- file.txt # give group 'interns' read-only
$ getfacl file.txt # view all ACL entries
$ setfacl -x u:alice file.txt # remove alice's specific entry
$ setfacl -b file.txt # remove ALL ACL entries
```

Note: A file with ACLs shows a '+' after its permission string in 'ls -l', e.g. '-rw-rw-r--+'. Use getfacl to see the full breakdown.

8. Quick Cheat Sheet

Task	Command
View permissions	ls -l file
View directory permissions	ls -ld dir
Add execute for owner	chmod u+x file
Set exact octal permissions	chmod 644 file
Recursive permission change	chmod -R 755 dir/
Change owner	chown user file
Change owner + group	chown user:group file
Change group only	chgrp group file / chown :group file
View/set default mask	umask / umask 022
Set SUID	chmod u+s file / chmod 4### file
Set SGID	chmod g+s dir / chmod 2### dir
Set sticky bit	chmod +t dir / chmod 1### dir
Grant ACL to a user	setfacl -m u:name:rwx file
View ACLs	getfacl file

Practice tip: create a scratch file with 'touch test.txt', then experiment with each chmod command above and check the result with 'ls -l' after every change.